

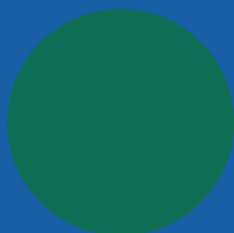


基于 MCP 协议的 x32dbg 自动化调试平台设计与实现

Design and Implementation of an Automated x32dbg Debugging
Platform Based on the Model Context Protocol

平台构建 · 可移植部署 · 性能优化 · 三个典型调试实验

Windows 10/11 x64 · WorkBuddy AI 客户端 · 完全可复现实验记录
2026 年 9 月



摘要

逆向分析与软件调试要求对目标进程的执行流进行精确、可重复的观测与干预。传统人工调试效率有限，而让 AI 助手直接驱动调试器又受制于接口的自动化程度。本文以 MCP（Model Context Protocol，模型上下文协议）为桥梁，设计并实现了一套由 AI 助手远程操控 x32dbg 调试器的自动化调试平台，并围绕该平台完成了三轮工程演进：第一代方案基于社区 HTTP 插件实现远程控制；第二代引入 x64dbg-automate 原生 ZeroMQ 消息通道，将三大调试实验的全流程耗时从 10 分钟以上压缩至约 2 秒；第三代接入官方 x64dbg-automate MCP 服务器，提供 48 个标准化调试工具，并交付了零硬编码路径、纯离线依赖、拷贝即用的一键分发包。

平台在自编演示程序 func_demo.exe 上完成三项实验验证：①断点跟踪与运行时栈观测（cdecl 约定下参数定位）；②运行时内存修改（参数 7 改写为 100，返回值 12 变为 105）；③二进制级语义补丁（7 字节改写将 add 变换为精确的 a - b 运算，输出 2）。实验结果与理论预期完全一致。本文同时系统归纳了实施过程中的关键技术决策与典型故障的定位方法，可直接指导同类平台的快速复现与迁移部署。

关键词：MCP；x32dbg；自动化调试；ZeroMQ；二进制补丁；可移植部署

目 录

目录	3
1. 引言与系统总体设计	4
1.1 整体架构图	4
1.2 本文完成的三项调试实验	4
2. 运行环境与软件准备	5
2.1 软件清单	5
3. 调试代理插件与依赖构建	5
3.1 安装 MCP 插件	5
3.2 零依赖桥接：不再需要 pip	6
4. MCP 客户端接入配置	6
5. 一键部署与可移植性设计	7
6. 实验一：断点跟踪与运行时栈观测	7
7. 实验二：运行时内存修改	7
8. 实验三：二进制级语义补丁	8
9. 常见问题与解决方案	8
10. 性能优化：从 HTTP 轮询到原生消息队列	9
11. 官方 MCP 服务器集成与一键分发包	9
11.1 最终架构	10
11.2 连接器的生成与信任机制	10
11.3 接入路上的两个新坑 (本次实战实录)	11
11.4 一键迁移包: 拷到任何机器 3 步开工	11
附录 A：MCP 插件 HTTP API 速查	12
附录 B：分发包文件清单（清理后）	12

1. 引言与系统总体设计

MCP（Model Context Protocol，模型上下文协议）是连接 AI 助手与外部工具的标准协议。将其引入逆向调试领域，可使 AI 助手直接驱动 x32dbg：下断点、单步执行、读写内存与寄存器、乃至实施二进制补丁。本文完整记录了该平台从零构建、两代方案演进到最终交付的全过程，所有步骤均在真实环境中验证可复现。

1.1 整体架构图

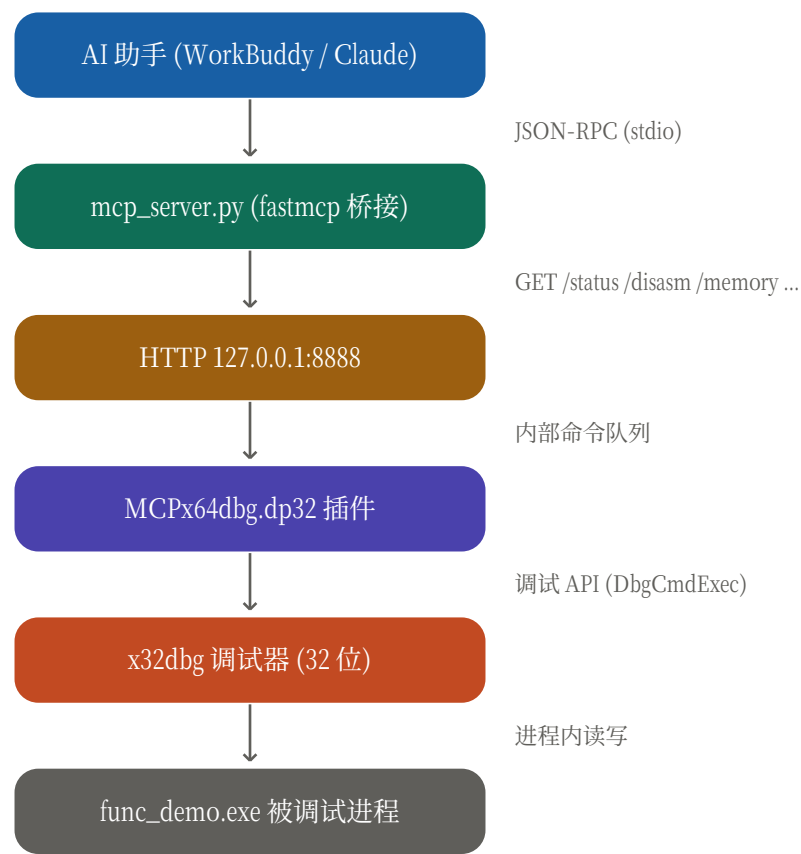
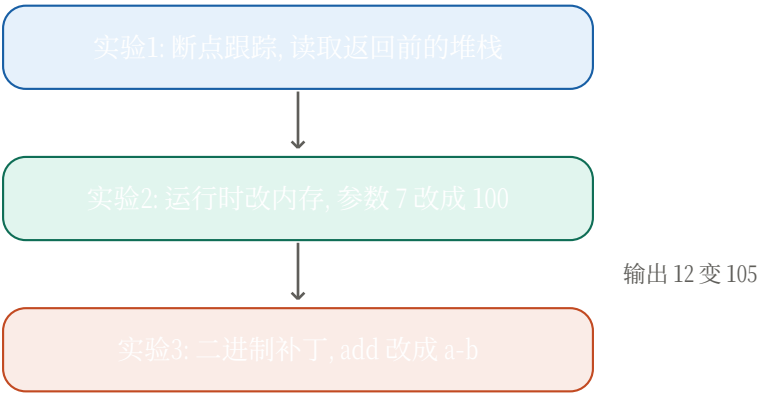


图 1-1 MCP 调试平台数据链路

要点：AI 通过 stdio 启动 mcp_server.py；该脚本把 MCP 调用翻译成对 127.0.0.1:8888 的 HTTP GET 请求；8888 端口由 x32dbg 内的 MCPx64dbg.dp32 插件监听，最终落到 x32dbg 的调试 API 上。全链路都在本机，无外部依赖。

1.2 本文完成的三项调试实验



2. 运行环境与软件准备

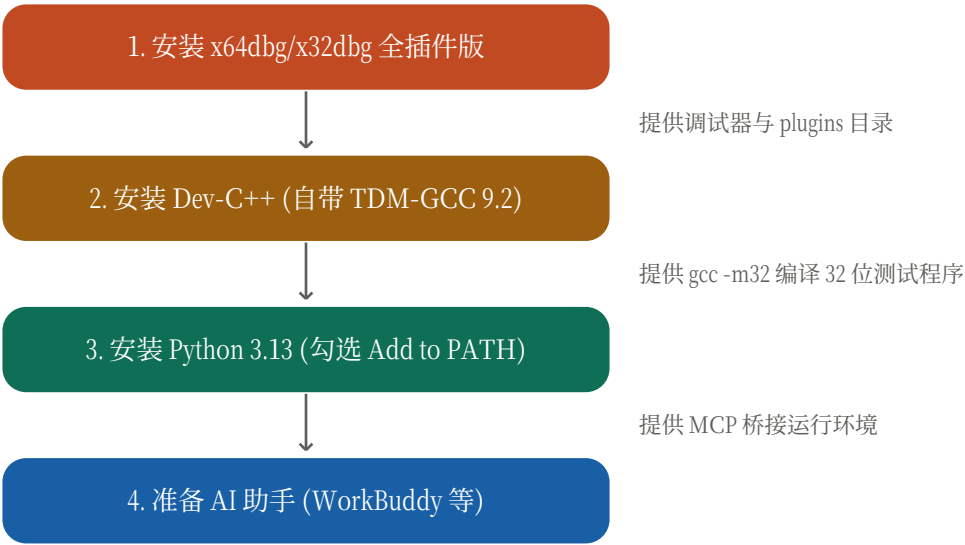


图 2-1 环境安装顺序

2.1 软件清单

软件	用途	本文实测版本/路径
x64dbg 全插件版	32/64 位调试器	F:\破解\x64dbg_2025-08-19全插件版\release\x32
Dev-C++ (Embarcadero)	C 编译器 TDM-GCC	...\Dev-Cpp\TDM-GCC-64\bin\gcc.exe (9.2.0, 支持 -m32)
Python (绿色版)	运行 MCP 桥接	3.12.10 嵌入式, 放在 E:\out\python312, 免安装免配置
MCPx64dbg.dp32	x32dbg MCP 插件	v1.0.x (社区开源, 129KB)

为什么必须 32 位？x32dbg 只能调试 32 位进程。编译时务必加 **-m32** 参数；TDM-GCC 9.2 自带 32 位多库支持（gcc -print-multi-lib 可验证）。

3. 调试代理插件与依赖构建

3.1 安装 MCP 插件

把 MCPx64dbg.dp32 复制到 x32dbg 的 plugins 目录即可，重启 x32dbg 后插件自动加载，并在 127.0.0.1:8888 开启 HTTP 服务：

```
rem 复制插件 (cmd)
copy MCPx64dbg.dp32 "F:\破解\x64dbg_2025-08-19全插件版\release\x32\plugins\"
rem 启动 x32dbg 后验证 (另开一个 cmd 窗口)
curl "http://127.0.0.1:8888/status"
rem 返回 {"version":3,"arch":"x32","debugging":true,"running":false} 即成功
```

3.2 零依赖桥接：不再需要 pip

初版方案采用 fastmcp 桥接，需要 pip 安装 fastmcp/requests，在国内网络下常被 403 反爬拦截，还需 socks5 代理引导 PySocks 自救，故障点多。当前 v3 方案已改为纯标准库实现（urllib + JSON-RPC over stdio，协议与 fastmcp 版完全兼容，48

个工具一一对应），整个包里没有任何第三方依赖——新机器上拷贝目录即可用，永远不需要 pip。

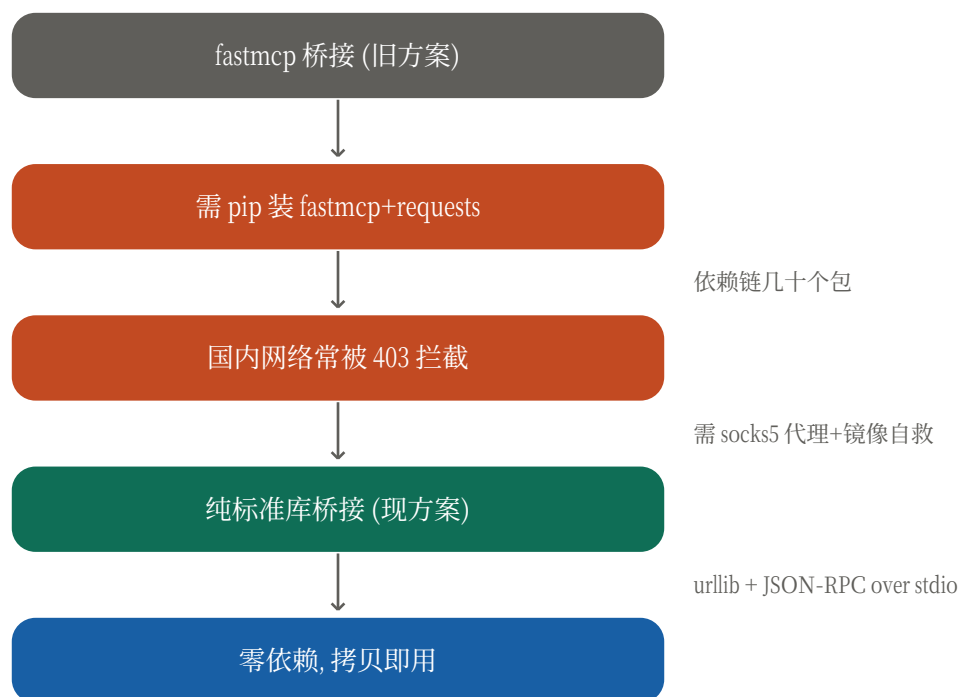


图 3-1 桥接方案演进：从 fastmcp 到零依赖

两个实现细节：① 用 urllib 时务必构造 ProxyHandler({}) 强制直连，否则系统代理会把 127.0.0.1:8888 的请求劫持成 502；② MCP 的 JSON-RPC 响应必须包在 "result" 字段里，直接摊平到顶层会导致客户端握手失败。

4. MCP 客户端接入配置

在 %USERPROFILE%\workbuddy\mcp.json（或对应 AI 客户端的 MCP 配置）中加入 x32dbg 服务器（一键搭建脚本会自动合并写入，无需手改；绝对路径按目标机实际解压目录现场生成）：

```
{
  "mcpServers": {
    "x32dbg": {
      "command": "<out>\\python312\\python.exe",
      "args": ["<out>\\mcp_server.py"]
    }
  }
}
```

保存后在 AI 客户端的连接器管理页找到 x32dbg 并点击"信任"。之后 AI 就拥有 40+ 个调试工具：get_status、execute_command、read_memory、write_memory、set_breakpoint、step_execution、disassemble_at、assemble_and_patch 等。

5. 一键部署与可移植性设计

配套文件 setup_mcp_onekey.bat（GBK 编码，双击即可）调用本目录绿色版 Python 运行搭建脚本，自动完成：相对路径探测 x32dbg → 检查插件 → 写 mcp.json → 生成 start_debug.bat。全程零依赖、零 pip、零管理员权限、不碰系统 Python。

```
@echo off
rem x32dbg MCP 一键搭建 v3 (零依赖, 使用本目录绿色版 Python)
set local
"%~dp0python312\python.exe" "%~dp0setup_mcp_onekey.py"
echo.
pause
```

setup_mcp_onekey.py 核心逻辑（完整文件随附于 E:\out）：

```
BASE = os.path.dirname(os.path.abspath(__file__)) # 一切路径以脚本目录为根
PY_EXE = BASE + r"\python312\python.exe" # 绿色版 Python
# 1. 优先相对路径: BASE\x64dbg\release\x32\x32dbg.exe
# 找不到再全盘限量搜索(深度5, 跳过系统目录)或提示手动输入
# 2. 检查 plugins\MCPx64dbg.dp32, 缺则从本目录复制(被占用则提示关 x32dbg)
# 3. 合并写 %USERPROFILE%\workbuddy\mcp.json
# 4. 以 GBK 编码生成 start_debug.bat (写入探测到的 x32dbg 路径)
```

GBK 关键点：中文 Windows cmd 默认代码页 936(GBK)。bat 里含中文路径时，文件必须以 GBK 编码保存；用 UTF-8 保存会导致路径乱码无法启动。便携性：把整个目录拷到任何 Windows 机器（python312 已在包内），双击 setup_mcp_onekey.bat 十秒完成搭建，然后双击 start_debug.bat 开始调试。

6. 实验一：断点跟踪与运行时栈观测

测试程序 func_demo.c 的 add 函数被编译在 0x401530。让 AI 在 add 入口下断点、单步执行，并停在 ret 处读取堆栈：

```
0x401530  push ebp                ; 保存 main 的栈帧
0x401531  mov  ebp, esp            ; 建立 add 的栈帧
0x401533  sub  esp, 0x10           ; 局部变量空间
0x401536  mov  edx, [ebp+0x08]      ; edx = a = 7
0x401539  mov  eax, [ebp+0x0C]      ; eax = b = 5
0x40153C  add  eax, edx            ; eax = 12
0x40153E  mov  [ebp-0x04], eax      ; result = eax
0x401541  mov  eax, [ebp-0x04]      ; 装载返回值
0x401544  leave                   ; 还原栈帧
0x401545  ret                     ; 返回 main
```

停在 ret (0x401545) 时实测堆栈：[esp]=0x004015A0(返回地址)、[esp+4]=7(参数a)、[esp+8]=5(参数b)、EAX=0xC(返回值12)。cdecl 调用约定下参数由调用者清理，返回前仍留在栈上。

7. 实验二：运行时内存修改

在 add 入口（求和指令执行之前）将栈上参数 a 由 7 改写为 100，观测返回值变化：

```
GET /register/get?name=esp -> 0x65FEAC (入口时 ESP)
```

```
GET /memory/write?addr=0x65FEB0&data=64000000 # a <- 100
GET /memory/read?addr=0x65FEB0&size=4 # 验证 64000000
GET /debug/step (x6) # eax = 0x69 = 105 !
GET /debug/run # 控制台输出 add(7,5)=105
```

两项关键结论：① 时机：补丁必须在 add eax,edx 执行之前，否则和已算好；② 定位：函数入口断点处 ebp 仍属调用者(main)的栈帧，参数地址=入口时 ESP+4，而非 EBP+8。

8. 实验三：二进制级语义补丁

仅改写 7 个字节，即可将 add 函数的语义变换为精确计算 $a - b$ ($7 - 5 = 2$)：

地址	改前字节	改后字节	改前指令	改后指令
0x401536	8B 55 08	8B 55 0C	mov edx,[ebp+8] (a)	mov edx,[ebp+0xC] (b)
0x401539	8B 45 0C	8B 45 08	mov eax,[ebp+0xC] (b)	mov eax,[ebp+8] (a)
0x40153C	01 D0	29 D0	add eax,edx	sub eax,edx

验证：断点停在 ret 时 EAX=0x2 ($7 - 5 = 2$)，控制台输出 add(7, 5) =

2。需注意：该补丁仅改写内存，进程重新加载后即消失；若需持久化，须将字节写回 EXE 文件。

9. 常见问题与解决方案

Q：程序停在陌生的 0x401820？

A：那是 MinGW 程序的 TLS 回调，x32dbg 默认在此暂停，再按一次运行即可，不是断点失效。

Q：AI 操作了但 x32dbg 界面不动？

A：插件从后台线程执行命令，不刷新 GUI。让 AI 每步补发一条 disasm eip 命令强制视图跟随 EIP。

Q：启动的 x32dbg 一闪就没了？

A：AI 会话结束时可能连带关闭子进程，应由用户双击 start_debug.bat 启动。

Q：pip 一直 403？

A：国内网络+安全拦截。用 socks5 代理+腾讯云镜像，或浏览器下 whl 本地安装（见第 3 章）。

Q：bat 中文乱码/无法启动？

A：bat 必须以 GBK 编码保存并 chcp 936；UTF-8 保存的中文路径会乱码。

Q：断点命中后寄存器是旧值？

A：确认 /status 返回 running:false（已暂停）再读寄存器；运行中读到的是瞬时值。

Q：装了 mcp 却报 No module named mcp.server.fastmcp？

A：mcp 2.x 把 FastMCP 改名成 MCPServer 且 API 大改，官方服务器只兼容 1.x。固定装 pip install "mcp<2"（实测 1.30.0）。

Q：wait_for_event 收到断点事件后程序却自己跑完了？

A：事件队列有积压，返回的可能是几步之前的陈旧事件（官方文档明示）。断点编排一律用 go() 后轮询状态/wait_until_stopped()，不要用 go+wait_for_event。

Q：自检脚本传 x96dbg.exe 报 Session did not appear？

A：x96dbg.exe 是启动器，真正调试器子进程的 PID 与 Popen 返回值对不上。先按目标 PE 位数解析成 x32dbg.exe/x64dbg.exe 再启动（官方 MCP 服务器内置该逻辑）。

Q：新机器怎么把整套平台搬过去？

A：整个 out 目录拷走（python312 依赖已装齐），双击 setup_xauto_onekey.bat，在 AI 连接器页信任 x64dbg-automate，最后跑 selftest_xauto.py 看全 PASS（见第 11 章）。

Q：连接器列表里看不到 x64dbg-automate？

A：mcp.json 路径或 JSON 格式有误。重跑 setup_xauto_onekey.bat 自动修复；手工修改过则检查双反斜杠转义，改完重启 AI 客户端再点信任。

Q：信任后调用工具报错/超时？

A：① 杀软拦截 python.exe 的本机回环通信（ZeroMQ 走 127.0.0.1），加白名单；② 有残留的 x32dbg 窗口占用会话，关掉后重试。

10. 性能优化：从 HTTP 轮询到原生消息队列

首版方案用 MCPx64dbg 插件的 HTTP 服务做远程调试, 实测暴露三大问题: ①插件的单线程迷你 HTTP 服务在连续请求下频繁掐断连接(WinError 10053/10061); ②为兜底而生的指数退避重试最坏一次要等 40 秒, 单步直播一步 4~20 秒; ③x32dbg 由脚本拉起时进程可能随会话回收而闪退, 应由用户双击 bat 启动。经社区调研, 迁移到官方推荐的 x64dbg-automate 方案: ZeroMQ REQ/REP 消息队列 + 完整会话管理, 客户端 pip install x64dbg-automate, 插件 x64dbg-automate.dp32 与 libzmq-mt-4_3_5.dll 一起放入 plugins。

操作	旧方案 (HTTP 插件)	新方案 (x64dbg-automate)
10 次单步直播	约 103 秒	0.1 秒
断点命中 + 读堆栈	数秒~40 秒(重试)	0.01 秒
三大实战全流程	超过 10 分钟且频繁中断	约 2.2 秒

迁移中的四个坑(血泪经验)：① 版本配对看协议代号——客户端 0.8.0 要 lilac_bonnet 协议插件, 0.9.x 配 ghost_fungus(即 0.8.1 插件), 不匹配会报 Incompatible plugin and client versions 或会话超时; ② libzmq-mt-4_3_5.dll 必须与 .dp32 同放 plugins 目录; ③ x32dbg 默认在 TLS 回调停一次, go() 后要循环 wait_until_stopped() 直到 EIP==目标地址; ④ 断点命中后 INT3 被摘除, 用 set_reg 重入函数前必须 clear_breakpoint() 再重新 set_breakpoint(), 否则断点不再生效。

```
# final_show.py 核心流程 (x64dbg-automate 官方 API)
client = X64DbgClient(x64dbg_path=X32DBG)
client.start_session(func_demo_exe)

client.go(); client.wait_until_stopped()           # 过系统断点
client.set_breakpoint(0x401530)                    # add 入口
client.go(); client.wait_until_stopped()           # 运行到断点
a = client.read_dword(esp + 4)                     # 读参数
client.write_memory(addr, b"\x64\x00\x00\x00")    # 内存补丁
client.stepi(1)                                    # 单步
client.disassemble_at(addr).instruction            # 反汇编
client.detach_session()
```

11. 官方 MCP 服务器集成与一键分发包

第 10 章的 final_show.py 以 Python 脚本直接调用 X64DbgClient；本章进一步将官方 x64dbg_automate 包自带的 MCP 服务器（48 个工具）注册为 WorkBuddy 连接器，AI 无需编写任何脚本，以自然语言即可驱动调试器。这是目前速度最快、工具最全、且随官方版本持续更新的最终形态。

11.1 最终架构



图 11-1 官方 MCP 服务器数据链路 (对比第 1 章的 HTTP 轮询方案)

11.2 连接器的生成与信任机制

MCP 连接器并非可以拷贝的文件，而是“配置条目 + 一次信任”的组合：配置条目写在 AI 客户端用户目录的 mcp.json 中（如 %USERPROFILE%\workbuddy\mcp.json），与机器绑定；换新机器后由一键脚本按该机器的实际路径重新生成，无需手动搬运。标准条目如下（一条配置通吃 32/64 位）：

```
{
  "mcpServers": {
    "x64dbg-automate": {
      "command": "<out>\\python312\\python.exe",
      "args": ["-m", "x64dbg_automate.mcp_server"],
      "env": { "X64DBG_PATH": "<out>\\x64dbg\\release\\x96dbg.exe" }
    }
  }
}
```

两个设计要点：① env X64DBG_PATH 指向 x96dbg.exe，服务器读取被调试目标的 PE 头后自动选择 x32dbg.exe 或 x64dbg.exe，32/64 位调试共用同一配置；② 以 -m 模块方式启动而非 Scripts 目录的 exe，规避绿色 Python 的 PATH 问题。

环节	操作	说明
生成-自动	双击 setup_xauto_onekey.bat	按目标机实际路径合并写入 mcp.json (推荐)

生成-手工	编辑 mcp.json 照样例填写	<out> 换成实际路径, 注意 JSON 双反斜杠转义
信任	AI 客户端连接器管理页 -> 自定义连接器 -> 信任 x64dbg-automate	每次修改 mcp.json 后需重启 AI 客户端
验证	对 AI 说"列一下调试会话"; 或跑 selftest_xauto.py	有会话列表/六步全 PASS 即链路可用

表 11-1 连接器生命周期四步操作速查

稳定性排查：① 连接器列表看不到条目 —— mcp.json 的路径或 JSON 格式有误，重跑一键脚本即可恢复；② 信任后调用报错 —— 杀毒软件拦截 python.exe 的本机回环通信(ZeroMQ 走 127.0.0.1)，将 python312 目录加入白名单；③ 会话超时 —— 先关闭多余的 x32dbg 窗口再重试，客户端会自动清理 Temp 下的陈旧会话锁文件。

11.3 接入路上的两个新坑 (本次实战实录)

坑一: mcp 2.x 改名 FastMCP — 首次 pip install mcp 装到了 2.2.0, 官方服务器 import 时报 No module named 'mcp.server.fastmcp': mcp 2.x 把 FastMCP 改名成 MCPServer 且 API 大改。必须 pin 旧版: pip install "mcp<2" (实测 1.30.0 正常, 48 个工具全部加载)。坑二: wait_for_event 会返回陈旧事件 — 事件队列会积压历史事件, wait_for_debug_event 官方文档明示"返回的事件可能发生在调用之前"。实战中: 收到 add 断点命中事件后以为还没到, 又 go 了一次, 把停在断点的程序放跑, 程序跑完退出, 之后读寄存器全 0 / 报 no debuggee。

```
# 错误编排 (实测翻车): go + wait_for_event 走断点
go(); wait_for_event('EVENT_BREAKPOINT')      # 收到的可能是几步之前的旧事件!
go()      # <- 多放跑一次, 程序直接运行到退出

# 正确编排 A (MCP 工具流): go 后轮询调试器状态
go(); 循环 get_debugger_status 直到 State=paused; 读 EIP 确认位置
# 正确编排 B (Python 客户端): go 后 wait_until_stopped()
client.go(); client.wait_until_stopped(); eip = client.get_reg('cip')
```

11.4 一键迁移包: 拷到任何机器 3 步开工

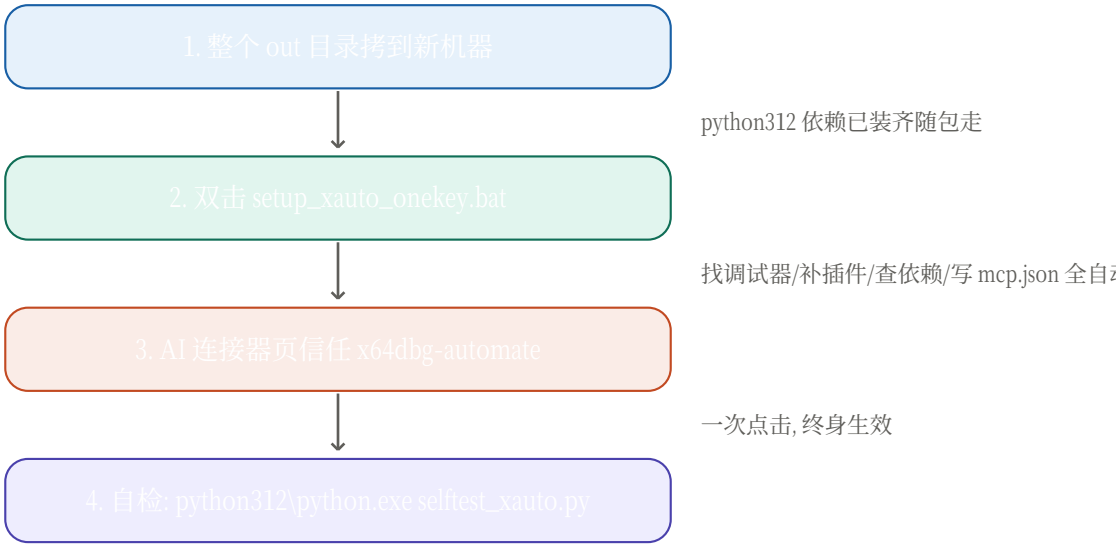


图 11-2 迁移流程 (全程零安装零管理员权限)

迁移包自检的坑: 直接给 X64DbgClient 传 x96dbg.exe 会 TimeoutError 'Session did not appear' —— x96dbg.exe 只是个启动器, 真正的调试器子进程 PID 对不上。必须像官方 MCP 服务器那样先按目标 PE 位数解析成 x32dbg.exe/x64dbg.exe 再启动 (selftest_xauto.py 已内置同款解析逻辑)。另外上一轮测试残留的 x32dbg 进程会占用会话锁文件, 客户端会自动清 Temp\xauto_session*.lock 陈旧锁, 但保险起见先关掉旧窗口。

```
# setup_xauto_onekey.py 做的事 (零硬编码, 一切以脚本目录为根)
BASE = os.path.dirname(os.path.abspath(__file__))
1. 探测调试器: BASE\x64dbg\release\x96dbg.exe -> 限量全盘搜索 -> 手动输入
2. 补齐插件: x64dbg-automate.dp32/.dp64 + libzmq-mt-4_3_5.dll -> plugins
   (从 tool\release32/64-0.8.1-ghost_fungus 备份自动复制)
3. 查依赖: x64dbg_automate/mcp/zmq/msgpack/pydantic/psutil
   缺则纯离线安装 tool\wheels (内置 py3.12+py3.13 全部 wheel, 不联网);
   离线不全再走腾讯云镜像直连 (国内可直接访问, 全程无需代理)
4. 合并写 ~/.workbuddy/mcp.json 的 x64dbg-automate 条目 (生成连接器)
5. GBK 生成 start_debug.bat / setup_xauto_onekey.bat
```

附录 A：MCP 插件 HTTP API 速查

端点	参数	说明
/status	-	调试器状态(版本/架构/是否运行)
/modules	-	已加载模块列表
/register/get	name=eip esp eax...	读寄存器
/register/set	name, value	写寄存器
/memory/read	addr, size(<=1024)	读内存(十六进制)
/memory/write	addr, data	写内存
/breakpoint/set	addr	下软件断点
/debug/run pause step stepover stepout	-	执行控制
/disasm	addr	反汇编单条指令
/cmd	cmd	执行任意 x32dbg 命令(bp/bpc/init/g...)
/assembler/assemble_mem	addr, instruction	汇编并写内存

附录 B：分发包文件清单（清理后）

文件 / 目录	说明
x64dbg\	x32dbg/x64dbg 全套 (plugins 已含 automate 插件+libzmq), 相对路径直接可用
python312\	绿色版 Python 3.12.10 嵌入式, 依赖已装齐 (x64dbg_automate/mcp 1.30/pyzmq...)
setup_xauto_onekey.py / .bat	一键搭建: 找调试器/补插件/离线装依赖/生成连接器配置/GBK bat
selftest_xauto.py	一键自检: 启动->符号解析->断点->读堆栈(7和15)->单步 全 PASS
start_debug.bat	启动 x32dbg 并加载 func_demo.exe (搭建时自动生成)
func_demo.c / func_demo.exe	32 位演示程序及源码 (add/factorial)
tool\release32/64-0.8.1-ghost_fungus\	automate 插件解压备份 (缺件时自动复制回 plugins)
tool\wheels\	全部 Python 依赖离线 wheel (py3.12 + py3.13 双版本)
tool\legacy_old_scheme.zip / *.zip / VC_redist*.exe	第一代方案留底、官方客户端源码、插件原始包、VC 运行库
gen_pdf.py / gen_readme.py	本文 PDF 与使用说明的生成源码 (reportlab)

关于 tool 目录：删除它不影响日常调试运行（运行只依赖 python312、x64dbg 与两个 bat）；保留它则拥有断网/新机器/重装系统场景下的完整离线恢复能力，建议保留。

免责声明：本文所有操作均在自编演示程序上进行，仅用于学习调试技术。请勿用于未经授权软件的逆向或破解。